



计算机网络

Socket编程

徐敬东 张建忠

xujd@nankai.edu.cn

zhangjz@nankai.edu.cn

计算机网络与信息安全研究室

应用进程标识方法

- **Q:** 在一台计算机中，进行用什么进行标识？
- 为了发送和接收消息，进程必须具有一个**标识符**
- 主机拥有一个唯一的32位的IPv4地址（或128位的IPv6地址）
- **Q:** 利用主机的IP地址表示主机中的进程是否可以？
 - **A:** 不可以。一个主机中可能同时运行着多个进程。

常用应用对传输层的要求

应用	数据丢失	带宽	时延
文件传输	否	弹性	否
电子邮件	否	弹性	否
Web文档	可容忍	弹性	否
实时音视频	可容忍	音频: 5kbps-1Mbps 视频: 10kbps-5Mbps	是
可缓存音视频	可容忍	同上	是
交互游戏	可容忍	高于几kbps	是
即时消息	否	弹性	是或否

2.3 Socket编程 – 举例（数据报客户端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockClient = socket(AF_INET, SOCK_DGRAM, 0);

    SOCKADDR_IN addrSrv;

    sendto(sockClient, sendBuf, sendlen, 0, (SOCKADDR *)&addrSrv, len);

    recvfrom(sockClient, recvBuf, 50, 0, (SOCKADDR *)&addrSrv, &len);

    closesocket(sockClient);
    WSACleanup();
}
```

2.3 Socket编程 – 举例（数据报服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockSrv = socket(AF_INET, SOCK_DGRAM, 0);

    SOCKADDR_IN addrSrv;
    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    SOCKADDR_IN addrClient; //远程IP地址
    loop {
        recvfrom(sockSrv, recvBuf, 50, 0, (SOCKADDR *)&addrClient, &len);
        sendto(sockSrv, sendBuf, sendlen, 0, (SOCKADDR *) &addrClient, len);
    }

    closesocket(sockClient);
    WSACleanup();
}
```


2.3 Socket编程 – 举例（流式客户端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockClient = socket(AF_INET, SOCK_STREAM, 0);

    connect(sockClient, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    recv(sockClient, recvBuf, 50, 0);
    send(sockClient, "hello", sendlen, 0);

    closesocket(sockClient);
    WSACleanup();
}
```

2.3 Socket编程 – 举例（流式服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);

    SOCKET sockSrv = socket(AF_INET, SOCK_STREAM, 0);

    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));

    listen(sockSrv, 5);

    while (1)
    {
        SOCKET sockConn = accept(sockSrv, (SOCKADDR*)&addrClient, &len);
        send(sockConn, sendBuf, strlen, 0);
        recv(sockConn, recvBuf, 50, 0);
        closesocket(sockConn);
    }

    closesocket(sockSrv);
    WSACleanup();
}
```

2.3 Socket编程 – 举例（多线程流式服务端）



```
void main()
{
    WSStartup(wVersionRequested, &wsaData);
    SOCKET sockSrv = socket(AF_INET, SOCK_STREAM, 0);
    bind(sockSrv, (SOCKADDR*)&addrSrv, sizeof(SOCKADDR));
    listen(sockSrv, 5);
    while (1) {
        SOCKET sockConn = accept(sockSrv, (SOCKADDR*)&addrClient, &len);
        hThread = CreateThread(NULL, NULL, handlerRequest, LPVOID(sockConn), 0, &dwThreadId);
        CloseHandle(hThread);
    }
    closesocket(sockSrv);
    WSACleanup();
}

DWORD WINAPI handlerRequest(LPVOID lparam)
{
    SOCKET ClientSocket = (SOCKET) (LPVOID) lparam;
    send(ClientSocket, sendBuf, strlen, 0);
    recv(ClientSocket, recvBuf, 50, 0);
    closesocket(ClientSocket);
    return 0;
}
```


■ **socketserver模块**：网络服务器编程架构

- TCPServer类：针对TCP服务器编程
- UDPServer类：针对UDP服务器编程

■ **TCPServer类与UDPServer类的使用**

- 编写请求处理类
- 创建TCPServer或UDPServer对象
- 利用TCPServer类或UDPServer类的server_forever()方法进行多次循环处理

2.3 Socket编程 – Python Socket编程举例

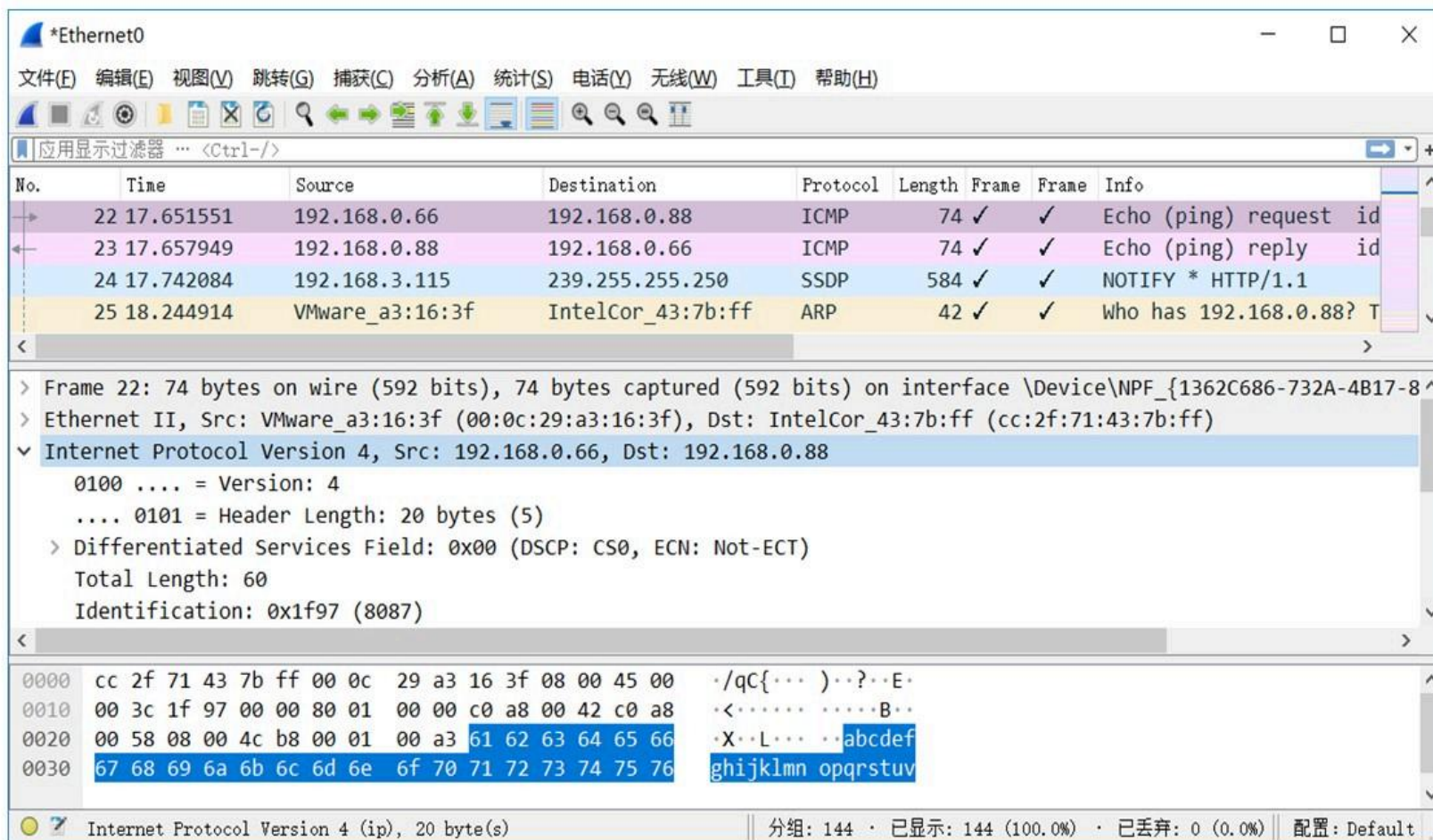


```
1 class MyTCPHandler(socketserver.BaseRequestHandler):
2     #重写基类中的handle函数
3     def handle(self):
4         #收发数据使用的socket存储在self.request中。
5         self.sock=self.request
6         .....
7         #接收数据。
8         self.recv_data=self.sock.recv(1024)
9         .....
10        #发送数据
11        self.sock.sendall(self.send_data)
12    .....
```

主程序

```
1     # 创建TCP服务器，为该服务器绑定的IP地址为host，端口号为port
2     with socketserver.TCPServer((host,int(port)), MyTCPHandler) as server:
3         # 循环进行收发处理，并在不用时自动关闭
4         server.serve_forever()
```

Wireshark



捕获数据包、设置显示过滤规则、设置捕获范围、查看统计信息

- **ping**: 测试网络连通性，通过发送ICMP回显请求与应答检测目标主机是否可达，并测量网络延迟、丢包率等
- **tracert**: 追踪数据包从本地计算机到目标地址所经过的路径，并记录每跳的延迟时间
- **ipconfig**: 查看IP地址、子网掩码、默认网关等基础网络信息
- **route**: 查看、添加、修改和删除路由表
- **arp**: 管理和查询主机的ARP缓存表
- **nslookup**: 查询域名系统记录，获取域名对应的IP地址或反向解析
- **netstat**: 显示网络连接、路由表及接口统计信息